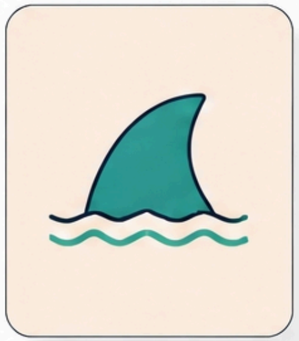
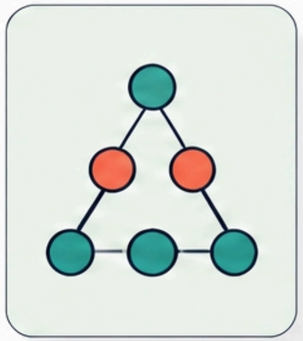
NETWORKING TOOLS HANDBOOK

Illustrated Tutorials · Running Examples · Complete Command Reference

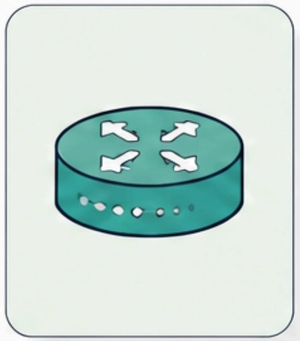


Wireshark

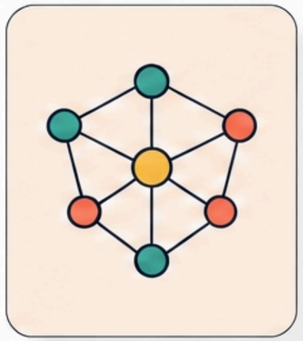


Mininet

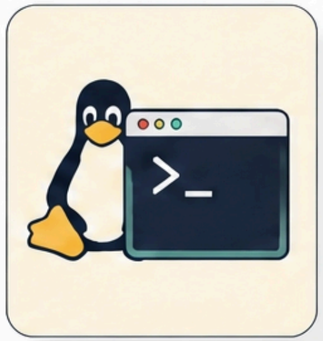
iperf3



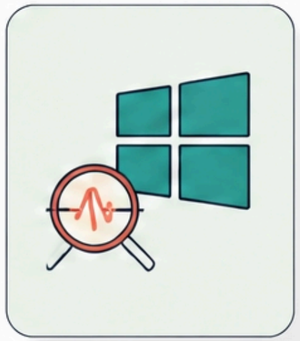
Cisco Packet Tracer



GNS3



Linux Utilities



Npcap

Computer Networks Lab by Nabajyoti Medhi

The Mental Model: Build → Measure → Capture

the engineer's workflow

1

BUILD TOPOLOGY

Packet Tracer · GNS3 · Mininet

2

MEASURE PERFORMANCE

iperf3 · ping · mtr

3

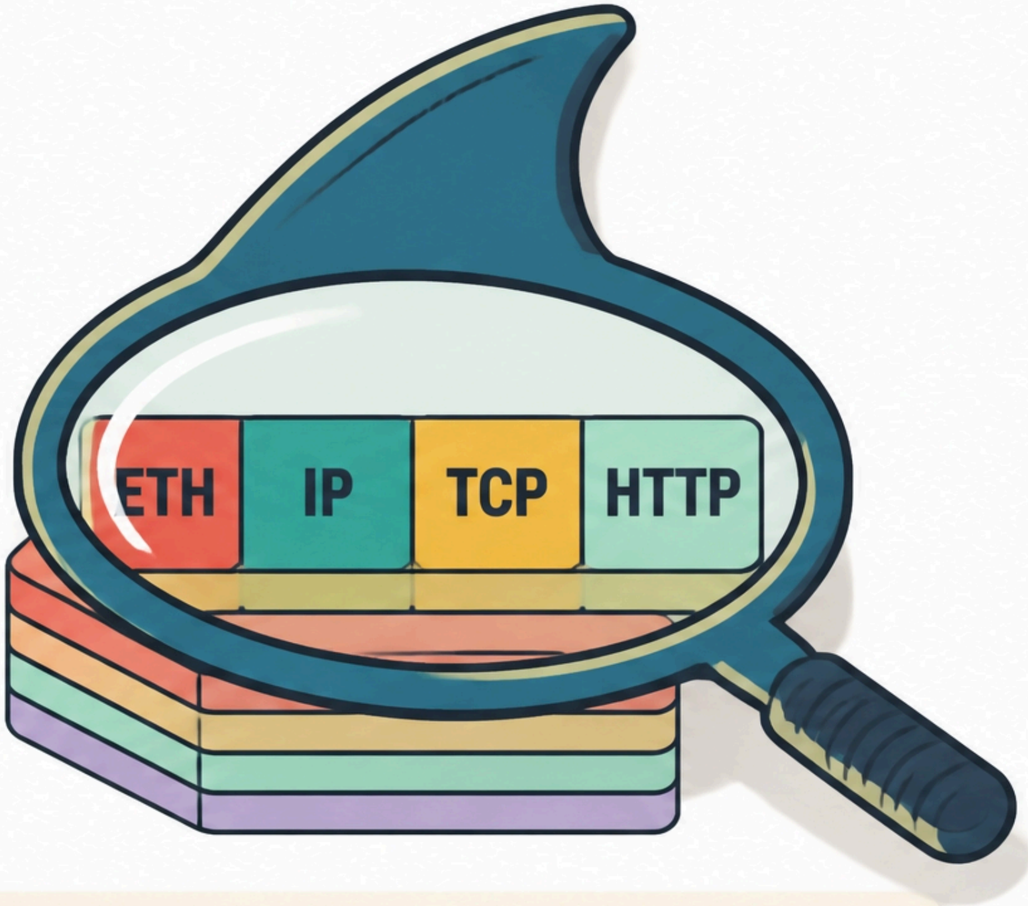
CAPTURE & TROUBLESHOOT

Wireshark · tcpdump · tshark

Start with topology building, move to measurement, then capture and troubleshood traffic.

SECTION 1 · WIRESHARK

Packet Capture and Protocol Analysis



4M+
downloads/year

Open source
since 1998

De-facto
industry standard

"What exactly happened on the wire?" — the question Wireshark was built to answer.

Wireshark Anatomy — The Three-Pane View

WIRESHARK

Filter bar (top)

Wireshark

Apply a display filter … <Ctrl-/>

1. Packet List

No.	Time	Source	Destination	Protocol	Info
1	0.000000	192.168.1.10	93.184.216.34	TCP	58012 → 80 [SYN] Seq=0 Win=64240
2	0.000145	93.184.216.34	192.168.1.10	TCP	80 → 58012 [SYN, ACK] Seq=0 Ack=1
3	0.000210	192.168.1.10	93.184.216.34	TCP	58012 → 80 [ACK] Seq=1 Ack=1
4	0.001012	192.168.1.10	93.184.216.34	HTTP	GET /index.html HTTP/1.1
5	0.034217	93.184.216.34	192.168.1.10	HTTP	HTTP/1.1 200 OK (text/html)
6	0.045678	192.168.1.10	8.8.8.8	DNS	Standard query 0x1a2b A www.example.com
7	0.046123	8.8.8.8	192.168.1.10	DNS	Standard query response 0x1a2b A 93.184.216.34
8	0.050256	192.168.1.10	192.168.1.1	ARP	Who has 192.168.1.1? Tell 192.168.1.10

Colorized protocols

2. Packet Details

```
▽ Frame 4: 512 bytes on wire (4096 bits), 512 bytes captured (4096 bits) on interface 0
▽ Ethernet II, Src: 192.168.1.10 (3c:52:82:aa:bb:cc), Dst: 93.184.216.34 (54:af:97:de:f0:11)
▽ Internet Protocol Version 4, Src: 192.168.1.10, Dst: 93.184.216.34
▽ Transmission Control Protocol, Src Port: 58012, Dst Port: 80, Seq: 1, Ack: 1, Len: 458
▽ Hypertext Transfer Protocol
    GET /index.html HTTP/1.1\r\n
    Host: www.example.com\r\n
    User-Agent: Mozilla/5.0\r\n
    \r\n
```

Click layer → highlights bytes

3. Packet Bytes

```
0000   3c 52 82 aa bb cc 54 af 97 de f0 11 08 00 45 00   <R....T.......E.
0010   02 08 1a 2b 40 00 40 06 b8 1c c0 a8 01 0a 5d b8   ...+@.@.......].
0020   d8 22 e2 fc 00 50 00 1c 2e e1 12 34 56 78 80 18   ."...P.....4Vx..
0030   fa f0 00 00 47 65 54 20 2f 69 6e 64 65 78 2e 68   ....GET /index.h
0040   74 6d 6c 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f   tml HTTP/1.1..Ho
0050   73 74 3a 20 77 77 77 2e 65 78 61 6d 70 6c 65 2e   st: www.example.
```

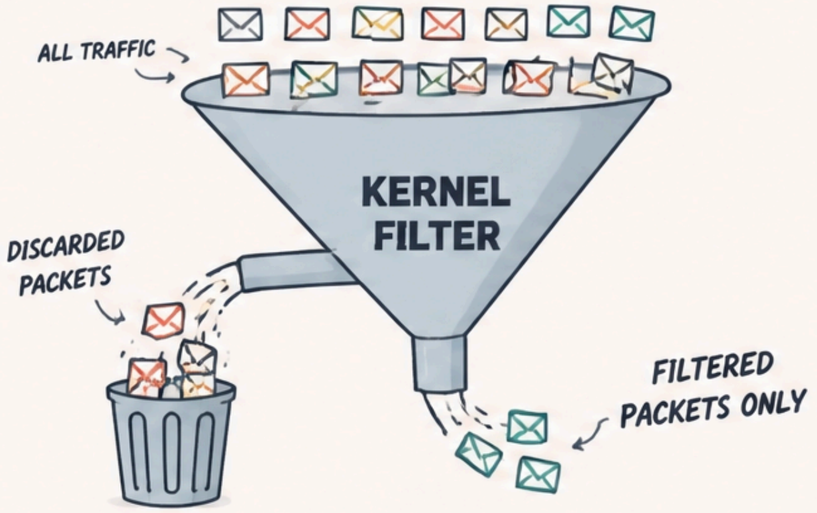
Follow TCP Stream (right-click)

Capture Filter vs Display Filter — Know the Difference

CAPTURE FILTER

BPF syntax · applied BEFORE capture

Capture filters decide at the kernel level which packets to keep.
Packets that don't match are discarded immediately — you cannot recover them later.




```
> host 192.168.1.10
> tcp port 80
> not broadcast and not multicast
```

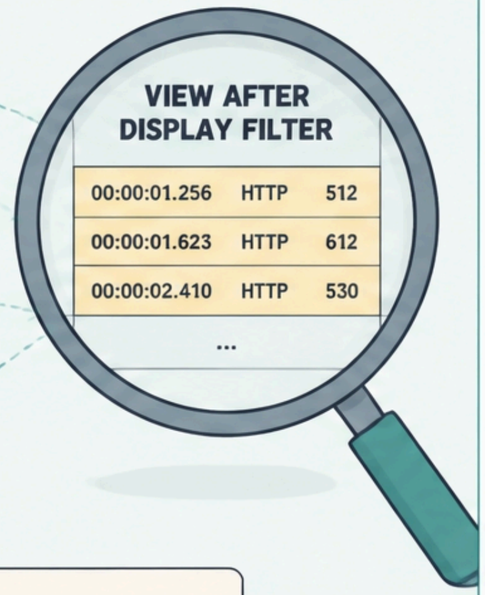
Master this distinction first!

DISPLAY FILTER

Wireshark syntax · applied AFTER capture

Display filters work on captured data.
All packets are still captured and available.
You're just choosing what to view.

FULL CAPTURED PACKET STACK				
00:00:01.123	192.168.1.5 → 10.0.0.1	TCP	74
00:00:01.130	10.0.0.1 → 192.168.1.5	TCP	66
00:00:01.256	192.168.1.10 → 93.184.216.34	HTTP	512
00:00:01.312	93.184.216.34 → 192.168.1.10	TCP	66
00:00:01.498	192.168.1.20 → 239.255.255.250	UDP	60
00:00:01.623	192.168.1.10 → 93.184.216.34	HTTP	612
00:00:01.945	192.168.1.15 → 8.8.8.8	DNS	75
00:00:02.102	93.184.216.34 → 192.168.1.10	TCP	66
00:00:02.410	192.168.1.10 → 93.184.216.34	HTTP	530
00:00:02.812	192.168.1.8 → 172.16.0.5	TCP	66
...			




```
> http.request.method == "POST"
> tcp.analysis.retransmission
> ip.addr == 192.168.1.0/24
```

Wireshark Display Filters · Layer 2 & Layer 3

Section 1.3

LAYER 2 (Ethernet, ARP, VLAN)

```
arp
eth.addr == 00:11:22:33:44:55
eth.dst == ff:ff:ff:f:f:ff:ff
vlan.id == 10
```

LAYER 3 (IP)

```
ip
ipv6
ip.src == 192.168.1.10
ip.dst == 192.168.1.20
ip.addr == 192.168.1..0/24
ip.ttl < 5
ip.flags.mf == 1  # fragmented
```

ICMP

```
icmp
icmp.type == 8  # echo request
icmp.type == 0  # echo reply
icmpv6
```

Tip: Wireshark autocompletes filter names — start typing and press TAB.

Wireshark Display Filters • TCP & UDP • The Detective's Toolkit

TCP

```
tcp
tcp.port == 443
tcp.srcport == 22
tcp.dstport in {80 443 8080}
tcp.flags.syn == 1 and tcp.flags.ack == 0   # SYN only
tcp.flags.reset == 1
tcp.analysis.retransmission
tcp.analysis.duplicate_ack
tcp.analysis.zero_window
tcp.analysis.out_of_order
tcp.window_size < 1000
```

The #1 filter for debugging slow connections

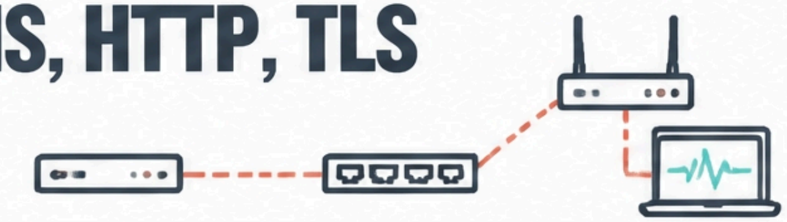
UDP

```
udp
udp.port == 53
```

BOOLEAN & REGEX

```
ip.src == 10.0.0.1 and tcp.port == 80
not arp
(http or dns) and not ip.addr == 192.168.1.1
frame contains "password"
frame matches "(?i)login"
```

Wireshark Display Filters · Application Layer · DNS, HTTP, TLS



DNS

```
dns
dns.flags.response == 0    # queries
dns.flags.response == 1    # responses
dns.qry.name contains "example"
dns.qry.type == 1          # A record
dns.flags.rcode != 0       # errors
```

HTTP

```
http
http.request
http.response
http.request.method == "POST"
http.response.code == 404
http.host contains "example.com"
http.user_agent contains "curl"
```

find the broken links!

TLS

```
tls
tls.handshake.type == 1    # ClientHello
tls.handshake.extensions_server_name
      == "example.com"
```

OTHER PROTOCOLS

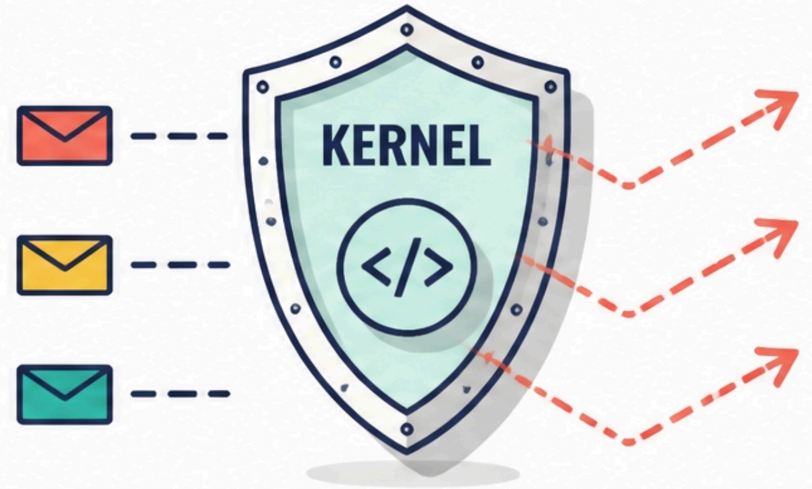
ssh · ftp · smtp · pop · imap · dhcp · ntp · snmp · radius · sip · rtp

Wireshark Capture Filters · BPF Syntax

Applied at the kernel — you cannot recover discarded packets

CAPTURE FILTER EXAMPLES

```
host 192.168.1.10
src host 10.0.0.5
dst host 10.0.0.5
net 192.168.0.0/24
net 10.0.0.0 mask 255.0.0.0
port 53
portrange 8000-9000
tcp port 80
udp port 53
icmp
arp
vlan 10
ether host 00:11:22:33:44:55
not broadcast and not multicast
(tcp port 80 or tcp port 443) and host 10.0.0.5
```



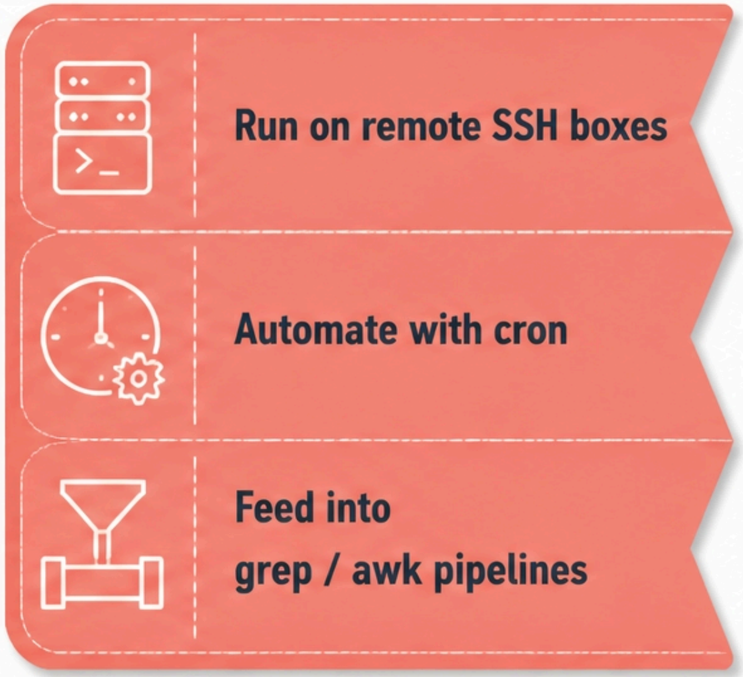
IMPORTANT:
BPF ≠ display filter syntax.
No dot-notation here.

tshark — Wireshark From the Command Line

Same engine, no GUI · perfect for scripts & remote servers

terminal

```
$ tshark -D                                  # list interfaces
$ tshark -i eth0                             # live capture
$ tshark -i eth0 -c 100                      # first 100 packets
$ tshark -i eth0 -w capture.pcap             # save to file
$ tshark -r capture.pcap                     # read from file
$ tshark -r capture.pcap -Y "http"           # display filter
$ tshark -r capture.pcap -Y "dns" \
    -T fields -e dns.qry.name -e ip.src
$ tshark -i eth0 -f "tcp port 80"            # capture filter
$ tshark -r capture.pcap -q -z io,stat,1     # I/O stats
$ tshark -r capture.pcap -q -z conv,tcp      # TCP conversations
$ tshark -r capture.pcap -q -z endpoints,ip  # endpoints (IP)
$ tshark -r capture.pcap -q -z http,tree     # HTTP summary tree
```



Section 6.1 · Linux `ip` — The Swiss Army Knife

Replaces the legacy ifconfig / route / arp commands

ADDRESSES

```
ip addr
ip -4 addr
ip addr show dev eth0
sudo ip addr add 192.168.1.10/24 dev eth0
sudo ip addr del 192.168.1.10/24 dev eth0
sudo ip addr flush dev eth0
```

LINKS

```
ip link
ip -s link
sudo ip link set eth0 up
sudo ip link set eth0 down
sudo ip link set eth0 mtu 1400
sudo ip link add veth0 type veth peer name veth1
sudo ip link add br0 type bridge
```

ROUTES

```
ip route
ip route get 8.8.8.8
sudo ip route add default via 192.168.1.1
sudo ip route add 10.0.0.0/24 via 192.168.1.254
sudo ip route del 10.0.0.0/24
```

NEIGHBORS & NAMESPACES

```
ip neigh
sudo ip neigh add 192.168.1.99 lladdr 00:11:22:33:44:55 dev eth0
sudo ip netns add lab1
sudo ip netns exec lab1 ip addr
sudo ip netns delete lab1
```

`ss` & `ping` — Socket Stats and Reachability

ss · Socket Statistics · Section 6.2

```
ss                      # all sockets
ss -a                   # incl. listening
ss -l                   # listening only
ss -t                   # TCP
ss -u                   # UDP
ss -n                   # numeric
ss -p                   # show process
ss -ltnp                # listening TCP w/ PIDs
ss -tunap               # everything
ss -s                   # summary of counts
ss -o state established #
ss -i                   # cwnd, rtt info
ss -tn dst 8.8.8.8
```

ss replaces the old netstat command

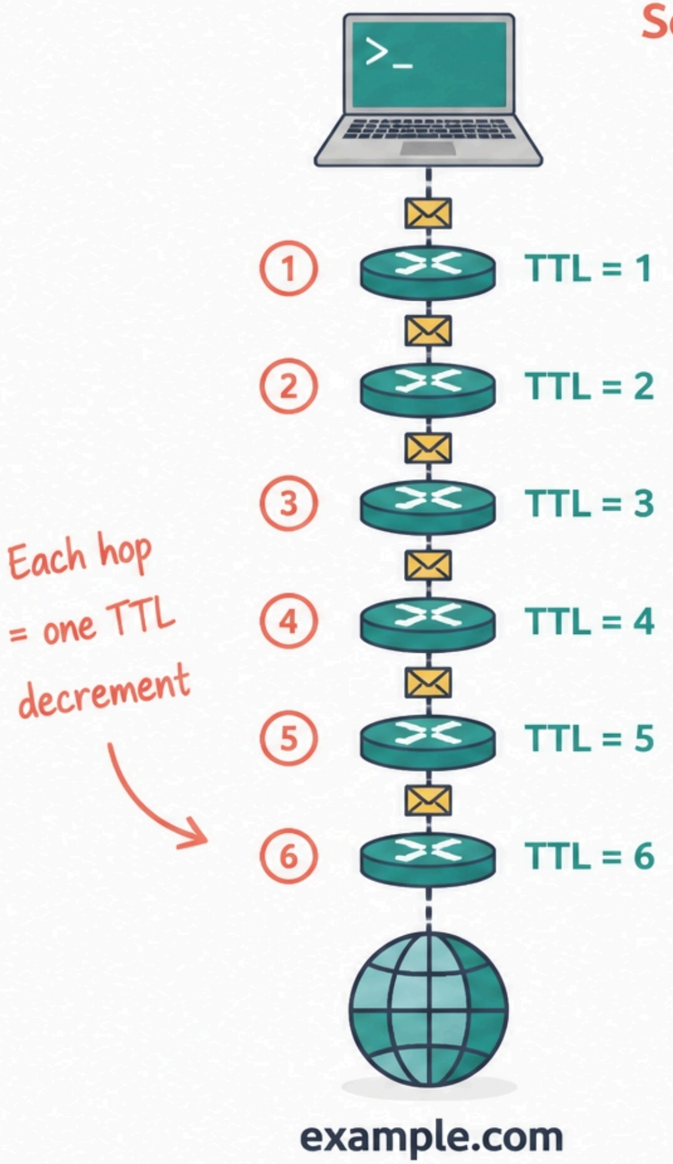
ping · Section 6.3

```
ping 8.8.8.8
ping -c 4 192.168.1.1          # count
ping -i 0.2  8.8.8.8           # fast interval (root)
ping -s 1400 8.8.8.8           # payload size (MTU test)
ping -M do -s 1472 8.8.8.8     # do not fragment
ping -I eth0 8.8.8.8           # source interface
ping -W 1 -c 5 host            # 1-sec timeout
ping -f host                   # flood (root)
ping6 ::1
ping -4 example.com
ping -6 example.com
```

discover the path MTU!

traceroute • mtr • tracepath — Path Discovery

Section 6.4 · See every hop between you and the destination



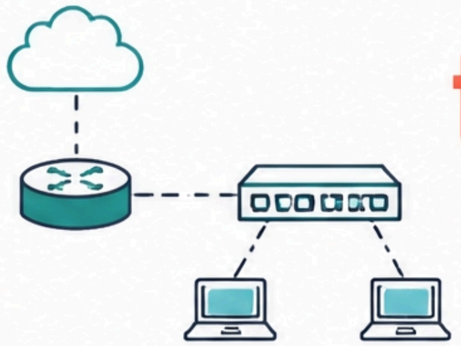
traceroute variants

```
traceroute example.com                  # UDP default
traceroute -I 8.8.8.8                   # ICMP
traceroute -T -p 443 example.com        # TCP SYN to 443
traceroute -n 1.1.1.1                   # numeric only
traceroute -m 20 host                   # max 20 hops
traceroute -q 1 host                    # 1 probe
traceroute -6 ipv6.google.com
```

tracepath & mtr

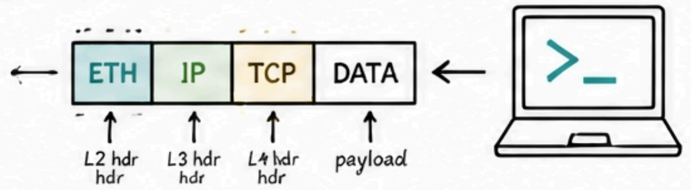
```
tracepath example.com                   # no root, discovers MTU
mtr example.com                         # continuous + ping
mtr -rwc 100 example.com                # report mode
mtr --tcp -P 443 example.com
```

mtr = traceroute + ping in one live dashboard — engineers love it.



tcpdump · The Original Packet Sniffer

Section 6.6 · Fast, scriptable, everywhere



BASICS

```
sudo tcpdump -D                    # list interfaces
sudo tcpdump -i eth0
sudo tcpdump -ni any
sudo tcpdump -c 20 -ni any         # first 20 packets
sudo tcpdump -v                    # verbose
sudo tcpdump -vvv -X               # hex + ASCII
sudo tcpdump -e                    # link header
```

FILTERS

```
sudo tcpdump -n 'host 192.168.1.50'
sudo tcpdump -n 'src host 10.0.0.5 and dst port 443'
sudo tcpdump -n 'port 53'
sudo tcpdump -n 'portrange 8000-9000'
sudo tcpdump -n icmp
sudo tcpdump -n arp
sudo tcpdump -n 'tcp[tcpflags] & (tcp-syn) != 0' # SYNs
```

SAVE & READ

```
sudo tcpdump -w out.pcap 'port 80'
sudo tcpdump -w cap-%Y%m%d-%H.pcap -G 3600 -Z root
tcpdump -r out.pcap
sudo tcpdump -w cap.pcap -C 100 -W 5  # ring buffer
sudo tcpdump -A -s 0 'tcp port 80'    # HTTP text
```

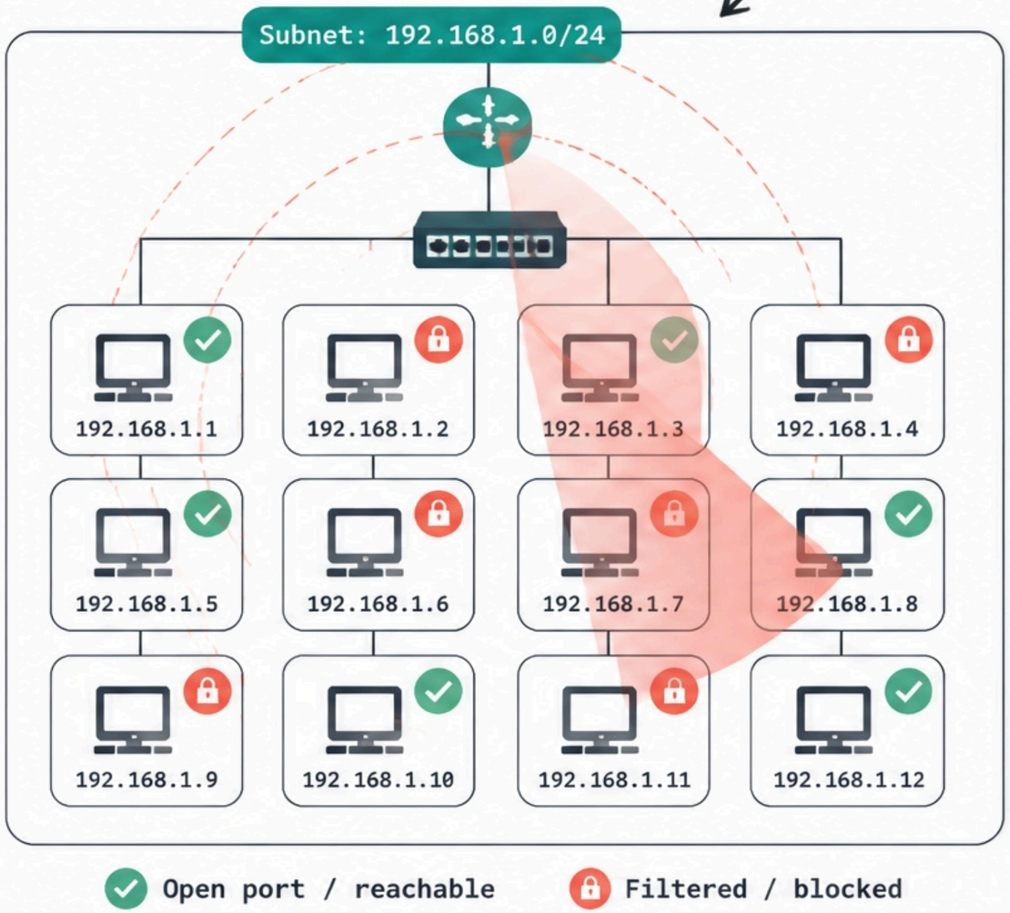
tcpdump writes standard pcap files — open them in Wireshark for deep inspection.

nmap · Network Discovery & Security Auditing

NMAP

Section 6.7 · The scanner everyone learns from

ONLy scan networks you own
or have permission for.



DISCOVERY & PORTS

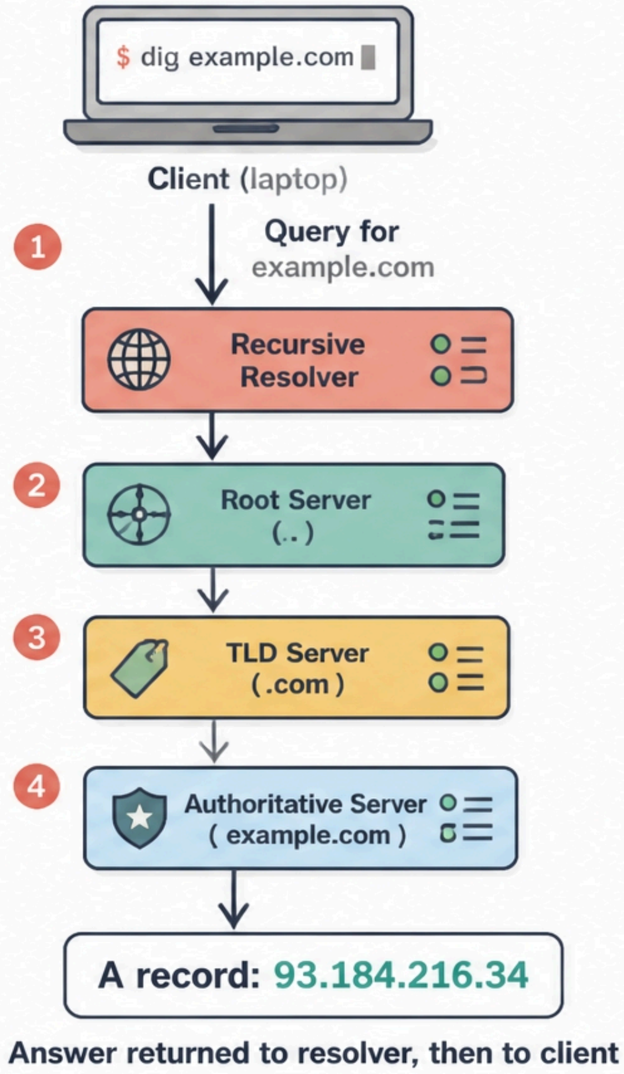
```
nmap 192.168.1.10                  # top 1000 TCP
nmap -sn 192.168.1.0/24            # ping sweep
nmap -F 192.168.1.10               # top 100 (fast)
nmap -p 22,80,443 host             # specific ports
nmap -p- host                      # all 65535 ports
```

SCAN TYPES & DETECTION

```
nmap -sS host                      # SYN scan (root)
nmap -sU -p 53,161 host            # UDP scan
nmap -sV host                      # service versions
nmap -O host                       # OS detection
nmap -A host                       # aggressive
nmap --script=default,vuln host    # NSE scripts
nmap -T4 -Pn host                  # skip ping
nmap -iL targets.txt               # from file
nmap -oA scan-report host          # save 3 formats
```

dig · host · nslookup — DNS Troubleshooting

Section 6.9 · Query the name system directly



dig — the pro tool

```
dig example.com
dig example.com A
dig example.com AAAA
dig example.com MX
dig example.com NS
dig example.com TXT
dig -x 8.8.8.8                    # reverse DNS
dig @8.8.8.8 example.com          # specific resolver
dig +trace example.com            # full recursion path
dig +short example.com
dig +noall +answer example.com
```

host

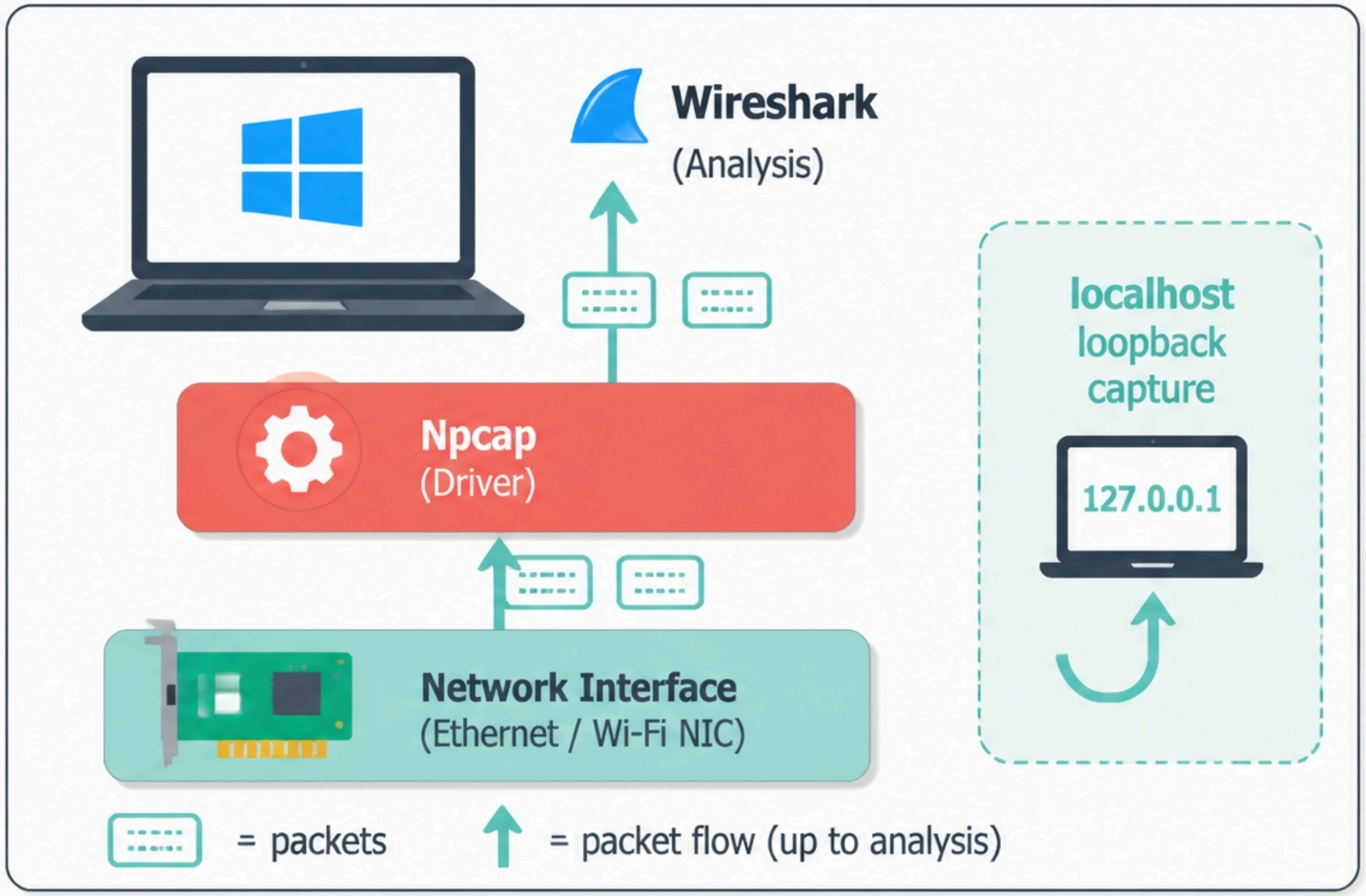
```
host example.com
host -t MX example.com
host -a example.com
```

nslookup

```
nslookup example.com
nslookup example.com 1.1.1.1
nslookup -type=MX example.com
```

Section 7 • Npcap • Windows Packet Capture

The driver that lets Wireshark see Windows traffic



1 INSTALL

Comes bundled with the Wireshark installer for Windows

2 WHAT IT ENABLES

- ✓ Live capture on Ethernet + Wi-Fi
- ✓ Loopback (127.0.0.1) capture
- ✓ Raw 802.11 monitor mode (some NICs)
- ✓ WinPcap-compatible API for legacy tools

3 COMMAND-LINE COUSIN

`WinDump.exe` — tcpdump-style CLI for Windows.

Modern replacement for the legacy WinPcap · maintained by the Nmap team

Section 8 · End-to-End Lab Recipes

Six hands-on labs to tie everything together

Learn. Build. Verify.

A LAB A · Packet Flow Correlation

Build a LAN in Packet Tracer · watch ARP + ICMP in Simulation · repeat on real host with Wireshark

Wireshark

```
1 ARP  Who has 192.168..2?
2 ARP  192.168!.2 is at ...
3 ICMP Echo (ping) request
4 ICMP Echo (ping) reply
...
```

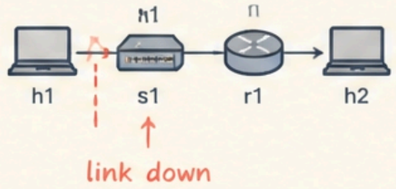
B LAB B · Throughput Explained

Run `iperf3 -c -P 4 -t 30` while tcpdump captures · inspect `tcp.analysis.retransmission` in Wireshark

```
# iperf3 client
iperf3 -c 10.0.0.2 -P 4 -t 30
# tcpdump capture
sudo tcpdump -i eth0 -w cap.pcap
```

C LAB C · Fault Injection

`sudo mn --link tc,bw=10,delay=10ms,loss=1` · then link s1 h1 down/up · observe recovery




```
$ sudo mn --link
tc,bw=10,delay=10ms,loss=1

$ link s1 h1 down
$ link s1 h1 up
```

observe recovery

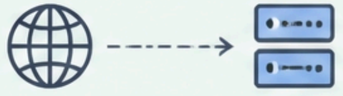
D LAB D · Static Routing (GNS3)

Two LANs, two routers · verify with **show ip route**, **ping**, **traceroute**

```
R1# show ip route
R1# ping 10.0.2.10
R1# traceroute 10.0.2.10
```

E LAB E · DNS Latency Study

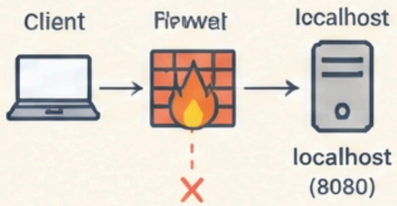
```
for i in $(seq 1 20); do
dig +noall +stats @8.8.8.8 example.com |
grep Query time; done
```



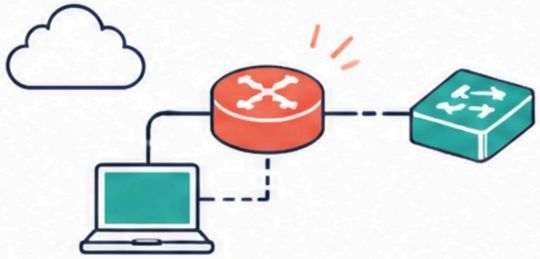

```
;; Query time: 12 msec
;; Query time: 13 msec
;; Query time: 11 msec
;; Query time: 15 msec
...
```

F LAB F · Firewall Verification

`sudo iptables -A INPUT -p tcp --dport 8080 -j DROP` · `nmap -p 8080 localhost` · compare open vs filtered

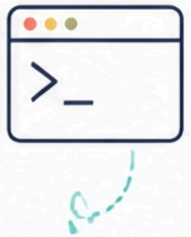



```
$ sudo iptables -A INPUT \
  -p tcp --dport 8080 -j DROP

$ nmap -p 8080 localhost

PORT     STATE
8080/tcp filtered
```



Section 9 . Compact Cheat Sheet

Everything on one page — print & pin above your desk



1. CAPTURE & ANALYSIS

```
wireshark &
tshark -i eth0 -Y "http"
sudo tcpdump -ni any port 53
capinfos capture.pcap
```

2. PERFORMANCE

```
iperf3 -s
iperf3 -c <server> -P 4 -t 30
iperf3 -c <server> -R
iperf3 -c <server> -u -b 50M
```

3. LINUX TOOLBOX

```
ip addr; ip route; ip neigh
ss -ltnp
ping -c 4 <host>
mtr <host>
dig <host>
sudo tcpdump -ni any
```

4. SIMULATION & EMULATION

```
# Packet Tracer: topology → IPs → Simulation
sudo mn --topo tree,depth=2,fanout=2 --test pingall

# GNS3: topology → CLI configure →
  capture on link
```

5. WINDOWS TOOLBOX

```
ipconfig /all
tracert 8.8.8.8
pathping google.com
Test-NetConnection example.com -Port 443
Get-NetTCPConnection -State Listen
```

6. DNS

```
dig +short example.com
dig -x 8.8..8.8
host -t MX example.com
dig +trace example.com
```

References & Acknowledgements

All content credited to original authors and open documenation

OFFICIAL DOCUMENTATION

1. **Wireshark User Guide & Display Filter Reference**
   wireshark.org
2. **Mininet Documentation**
   mininet.org
3. **iperf3 Manual**
   iperf.fr
4. **Cisco Packet Tracer**
   netacad.com
5. **GNS3 Documentation**
   docs.gns3.com
6. **Linux man pages**
   ip(8), ss(8), tcpdump(1), nmap(1), dig(1)
7. **Npcap Reference**
   npcap.com

BOOKS & COURSES

1. **Wireshark Network Analysis**
   Laura Chappell
2. **Computer Networking: A Top-Down Approach**
   Kurose & Ross
3. **TCP/IP Illustrated Vol.1**
   W. Richard Stevens
4. **Nmap Network Scanning**
   Gordon Lyon
5. **The Practice of System & Network Administration**
   Limoncelli et al.

> *This handbook synthesizes public documentation, open-source manuals, and classroom teaching notes. All trademarks belong to their respective owners.*

KEEP CAPTURING. KEEP LEARNING.

The wire never lies — and now you know how to listen.